

Project: document Q&A agent with memory

LESSON 06 · KNOWLEDGE & RAG

What you'll learn

- Assemble a single support agent that answers from your documents and remembers each customer — using only pieces you already built across this course
- Configure a live-conversation agent: keep the default model for speed, attach a knowledge base, and turn on memory with cross-session recall
- Prove cross-session memory by telling the agent something in one chat and having it use that fact in a brand-new session
- Read the trace timeline to confirm a single answer combined a document-embedding (RAG) step and a search-memory step
- Articulate the four deliberate capabilities — model, memory, cross-session, agentic RAG — that make this an architected agent rather than one you just spun up

Key concepts

This capstone assembles parts you already built — you create nothing new, you just connect them. Over the previous lessons you picked a model, turned on memory, and built a knowledge base. The build here is fast precisely because every piece already exists; the lesson is about composing them into one agent that answers from docs and remembers the customer.

Keep the default model when speed matters, because this is a live conversation. A document Q&A agent is answering a customer in real time, so latency is part of the product. The instructor deliberately keeps the default model rather than reaching for a heavier one — the reasoning engine just needs to be quick and good enough for grounded support answers.

Memory and RAG are two different mechanisms, and a good agent uses both in one answer. Memory holds who the customer is and what they told you; the knowledge base holds the facts in your documents. When the agent is asked whether the warranty covers dead pixels, it pulls the warranty terms from the documents *and* recalls from memory which monitor the customer owns — neither alone could answer the question.

Cross-session memory is what lets a customer skip the repetition when they come back. Turning on cross-session (the transcript's "cross-section") means the agent remembers a customer across separate conversations, not just within one chat. The real test is

TAKEAWAY

This capstone assembles parts you already built — you create nothing new, you just connect them.

TAKEAWAY

Keep the default model when speed matters, because this is a live conversation.

TAKEAWAY

Memory and RAG are two different mechanisms, and a good agent uses both in one answer.

TAKEAWAY

Cross-session memory is what lets a customer skip the repetition when they come back.

starting a completely new session, asking about "my monitor" without naming it, and having the agent still know it's a Northview 27 from a previous conversation.

The trace is your proof that retrieval and memory actually fired — don't take the answer on faith. Opening the timeline shows the agent did two distinct things to answer one question: a document-embedding step that went to the knowledge base for warranty terms, and a search-memory step that recalled who the customer is. Expanding the memory step reveals exactly what was pulled back, which is how you verify the architecture is working as designed.

TAKEAWAY

The trace is your proof that retrieval and memory actually fired — don't take the answer on faith.

In Studio

Primary surface: **Knowledge** → **Knowledge Base** and **Connections** → **Memory**, assembled on a single agent.

1. **Create one agent** and give it a role — here, a Northwind support assistant. You are not building anything new; you are wiring up existing pieces.
2. **Keep the default model.** Because this is a live conversation, leave the model as-is so responses stay fast.
3. **Attach the knowledge base** you built in the earlier lesson so the agent answers from your real documents (agentic RAG).
4. **Turn on memory** for the agent so it can remember what a customer tells it.
5. **Switch on cross-session memory** so the agent carries a customer's context across separate conversations, not just within a single chat.
6. **Introduce yourself like a customer** in the first chat — e.g. "I'm Felipe, and I bought a Northview 27." The agent confirms it will remember; it isn't looking anything up yet, it's just storing the fact.
7. **Open a completely new chat / session** — a clean slate that would normally make an agent forget — and ask a question that depends on the earlier fact without restating it, e.g. "Does my monitor's warranty cover dead pixels?"
8. **Read the answer:** the agent both recalls the monitor is a Northview 27 (memory) and pulls the actual warranty terms (documents) in one response.
9. **Open the trace timeline** to verify. You'll see a **document-embedding** step (knowledge base lookup) and a **search-memory** step (recalling the customer). Expand the memory step to see exactly what it pulled back from the previous conversation.

TRY THIS

- 1 Rebuild this agent end to end from your own pieces: create a support agent, attach a knowledge base you already made, keep the default model, and turn on memory with cross-session enabled. Resist adding anything new.
- 2 Run the cross-session test deliberately. In one chat, tell the agent a fact about yourself or your product; then start a *fresh* session and ask a question that only makes sense if it remembers that fact — without restating it. Confirm the answer uses both the remembered fact and a document.
- 3 Open the trace on that answer and find the two steps — the document-embedding step and the search-memory step. Expand the memory step and read what was retrieved, so you can prove RAG and memory each fired in a single response.